Foundations of Natural Language Processing

A Practical Guide to Text Analytics with Python

Dip Das

October 20, 2026

Abstract

Natural Language Processing (NLP) is a branch of artificial intelligence that empowers computers to understand, interpret, and manipulate human language. This document serves as a practical foundation for building NLP pipelines in Python, covering preprocessing, vectorization, embeddings, and modern transformer models.

1 Decoding Human Language

Unlike structured databases, human text is messy, ambiguous, and context-dependent. The primary goal of an NLP pipeline is to transform unstructured text into mathematical representations that machine learning algorithms can digest.

A standard NLP pipeline typically follows these steps:

1. **Raw Text Collection:** Gathering string data.
2. **Tokenization:** Splitting text into discrete units.
3. **Vectorization:** Converting text tokens into numerical arrays.
4. **ML Modeling:** Training classifiers or generative models.

2 Text Preprocessing

Before counting words or training models, we must clean the data. **Tokenization** splits a string into individual units (tokens), such as words or subwords. Following this, we often apply lowercasing to unify vocabulary, remove punctuation, and discard **Stop Words**—common words like “the”, “is”, and “in” that carry little predictive signal for most downstream tasks.

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords

# Download required NLTK data (run once)
nltk.download('punkt')
nltk.download('stopwords')

text = "NLP is incredibly fascinating, isn't it?"
tokens = word_tokenize(text.lower())

# Filter out standard English stop words
stop_words = set(stopwords.words('english'))
clean_tokens = [w for w in tokens if w.isalpha() and w not in stop_words]
```

```
print(clean_tokens)
# Output: ['nlp', 'incredibly', 'fascinating']
```

Listing 1: Basic Tokenization with NLTK

3 Feature Extraction (Vectorization)

Machine learning models require numerical vectors. The **Bag of Words (BoW)** approach creates a vocabulary from the entire dataset and counts the frequency of each word per document. However, BoW treats all words equally. To solve this, we use **TF-IDF** (Term Frequency - Inverse Document Frequency).

TF-IDF weighs words by how frequently they appear in a specific document (TF), offset by how common they are across the entire corpus (IDF).

Table 1: Example Bag of Words Word Counts

Document	**cat**	**dog**	**chased**	**the**
1. “the cat chased the dog”	1	1	1	2
2. “the dog”	0	1	0	1
3. “the cat”	1	0	0	1

```
from sklearn.feature_extraction.text import TfidfVectorizer

documents = [
    "The cat chased the dog",
    "The dog",
    "The cat"
]

vectorizer = TfidfVectorizer(stop_words='english')
tfidf_matrix = vectorizer.fit_transform(documents)

print(vectorizer.get_feature_names_out()) # ['cat' 'chased' 'dog']
print(tfidf_matrix.shape)                 # (3, 3) Sparse Matrix
```

Listing 2: TF-IDF Vectorization with Scikit-Learn

4 Word Embeddings & Semantic Meaning

While TF-IDF is computationally efficient, it lacks semantic understanding; it treats “car” and “automobile” as orthogonal (unrelated) features. **Word Embeddings** (like Word2Vec or GloVe) solve this by mapping words into a dense, low-dimensional mathematical space.

In this space, words with similar meanings are positioned closer together. Because they are represented as continuous vectors, we can perform semantic arithmetic. The classic example is:

$$\text{King} - \text{Man} + \text{Woman} \approx \text{Queen}$$

5 The Transformer Era

Modern NLP is dominated by the **Transformer** architecture and Large Language Models (LLMs) like BERT and GPT (Vaswani et al., 2017). Transformers rely on the *Self-Attention Mechanism*, which allows the model to look at every word in a sentence simultaneously and weigh the importance of each word's relationship to every other word, granting unprecedented contextual awareness.

```
from transformers import pipeline

classifier = pipeline("sentiment-analysis")

results = classifier([
    "I absolutely love learning about natural language processing!",
    "Debugging this code has been extremely frustrating."
])

for r in results:
    print(f"{r['label']} (Score: {r['score']:.4f})")
# Output: POSITIVE (0.999), NEGATIVE (0.998)
```

Listing 3: Using Transformers via Hugging Face

6 Practical Project: Text Classification

Let's combine our knowledge to build an end-to-end text classifier. We will use a pipeline that chains our TF-IDF vectorizer directly into a Multinomial Naive Bayes classifier.

```
from sklearn.pipeline import make_pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB

# 1. Prepare minimal training data
X_train = [
    "Free money, click here now",
    "Win a free gift card today",
    "Meeting schedule for tomorrow at 10am",
    "Can we review the project specs?"
]
y_train = ["spam", "spam", "ham", "ham"]

# 2. Build and train the Scikit-Learn Pipeline
model = make_pipeline(
    TfidfVectorizer(lowercase=True, stop_words='english'),
    MultinomialNB()
)
model.fit(X_train, y_train)

# 3. Predict on unseen data
X_test = ["Claim your free prize now!", "Please send the report"]
predictions = model.predict(X_test)

for text, pred in zip(X_test, predictions):
    print(f"'{text}' -> {pred.upper()}")
```

Listing 4: End-to-End Text Classification